

PEC 1: Búsqueda y Juegos

Presentación

Primera PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos.

Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas informadas (formalización y estrategias relacionadas) y juegos (minimax y poda alfa-beta).

Descripción de la PEC/práctica a realizar

En esta PEC sencillamente queremos que se ponga en práctica lo que se ha aprendido sobre los principales algoritmos de esta primera parte del curso: Búsqueda y Juegos

Algoritmos de búsqueda:

1.- (2 puntos) Si la heurística es admisible pero no consistente, al ejecutar el A*, ¿pueden volver a aparecer en *Abiertos* nodos que ya están en *Cerrados*?

Proponed un grafo y un heurístico que ejemplifique la respuesta que se ha dado (*no sirven los ejemplos de los materiales, quiero que os lo penséis vosotros; se pueden encontrar grafos pequeños que sirven de ejemplo*). Hacedlo muy detallado: Argumentad por qué el heurístico es o no admisible, es o no consistente y realizad una ejecución manual del A* (enseñando a cada paso los *Abiertos* y los *Cerrados*).

Solución:

Sí. La consistencia (o monotonía) garantiza que el valor de $f(n)$ nunca decrece a lo largo de un camino. Si la heurística es sólo admisible, podríamos encontrar un camino hacia un nodo n que parezca óptimo en un momento dado, pero más tarde encontrar un camino alternativo hacia el mismo nodo con un coste $g(n)$ menor. Si no reabrimos el nodo (es decir, si no vuelve a aparecer en la lista *Abiertos* para actualizar a sus vecinos), el A* podría acabar devolviendo una solución subóptima.

Ejemplo pedido:

Sea el grafo definido de la siguiente manera: Tenemos cuatro nodos **A**, **B**, **C**, y **D** donde

- **Arestas y Costes:**
 - $A \rightarrow B$ con coste 1
 - $A \rightarrow C$ con coste 10
 - $B \rightarrow C$ con coste 1
 - $C \rightarrow D$ con coste 100
- **Valores Heurísticos $h(n)$:**
 - $h(A) = 100$
 - $h(B) = 100$
 - $h(C) = 0$
 - $h(D) = 0$

El nodo inicial es **A** y el nodo objetivo es **D**. El heurístico es admisible ya que:

- $100 = h(A) \leq h^*(A) = 102$
- $100 = h(B) \leq h^*(B) = 101$
- $0 = h(C) \leq h^*(C) = 100$
- $0 = h(D) \leq h^*(D) = 0$

pero no es consistente, ya que debería pasar que $100 = h(B) \leq \text{coste}(B,C) + h(C) = 1+0=1$, y ¡esta claro que $100 \leq 1$ es falso!

Así, si realizamos una búsqueda A* con nodo inicial **A** y nodo objetivo **D**:

1. Abiertos: [['A', 100]]	Cerrados: []
2. Abiertos: [['C', 10.0], ['B', 101.0]]	Cerrados: [['A', 100]]
3. Abiertos: [['B', 101.0], ['D', 110.0]]	Cerrados: [['C', 10.0], ['A', 100]]
4. Abiertos: [['C', 2.0], ['D', 110.0]]	Cerrados: [['B', 101.0], ['C', 10.0], ['A', 100]]
5. Abiertos: [['D', 102.0], ['D', 110.0]]	Cerrados: [['C', 2.0], ['B', 101.0], ['C', 10.0], ['A', 100]]

El resultado final es el camino **A → B → C → D** de coste **102**. Fijémonos que al pasar del paso 3 al paso 4 'C' vuelve a entrar en la lista de *Abiertos*.

El ejemplo demuestra que la consistencia es una propiedad más fuerte que la admisibilidad. Si la heurística es consistente, A* encuentra el camino óptimo hacia cualquier nodo la primera vez que lo explora. Si sólo es admisible, el algoritmo puede rectificar y reexplorar nodos para garantizar el óptimo final.

2.- (1 punto) En el algoritmo A*, ¿cómo afecta la dominancia de una heurística h2 sobre otra heurística h1 al número de nodos expandidos?

Solución:

El algoritmo con h2 expandirá, como máximo, el mismo número de nodos que con h1, pero normalmente expandirá menos.

En el algoritmo A*, se expanden todos los nodos donde $f(n) < C^*$ (siendo C^* el coste de la solución óptima). Dado que $h_2(n) \geq h_1(n)$, el valor $f_2(n) = g(n) + h_2(n)$ será más cercano a C^* que $f_1(n)$. Esto hace que el "frente de búsqueda" sea más dirigido y se exploren menos caminos irrelevantes que no llevan al objetivo.

3.- (1 punto) ¿Qué pasa si cambiamos el algoritmo A* y consideramos un factor constante que multiplique el heurístico: $f(n) = g(n) + W \cdot h(n)$ con $W > 1$?

Solución:

La búsqueda es mucho más rápida (expande menos nodos) pero se pierde la garantía de optimalidad.

Al dar más peso a la heurística, el algoritmo se comporta de manera más "voraz" (*greedy*), priorizando llegar rápidamente a la meta por encima de encontrar el camino más corto. La solución encontrada estará acotada: su coste nunca será superior a W veces el coste de la solución óptima ($W \cdot C^*$). Es una técnica muy utilizada cuando el espacio de estados es demasiado grande para un A* estándar.

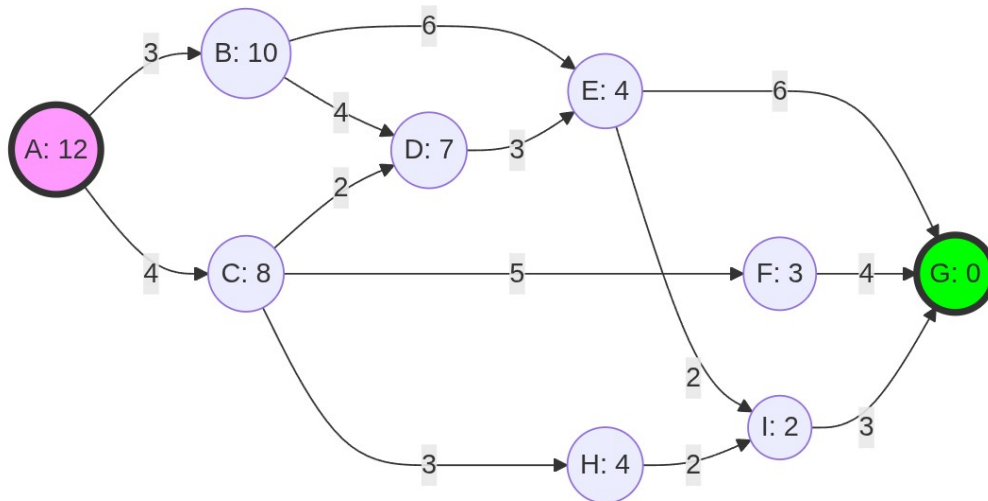
4.- (1 punto) ¿Cuál sería el comportamiento del A* si tuviéramos $h(n) = h^*(n)$ (el coste real exacto)?

Solución:

A* iría *directo como una flecha* hacia el objetivo, expandiendo sólo los nodos que forman parte del camino óptimo.

Para cualquier nodo n en un camino óptimo, tendremos que $f(n) = g(n) + h^*(n) = C^*$. Para cualquier nodo fuera del camino óptimo, tenemos que $f(n) > C^*$. Dado que el A* siempre expande primero los nodos con la f más pequeña, elegirá exclusivamente los nodos del camino de interés y se detendrá en cuanto llegue a la meta, sin explorar ninguna rama lateral.

5.- (3 puntos) Dado el grafo dirigido:



con el coste detallado en las aristas y la función heurística detallada en los nodos.

Utilizad la **búsqueda ávida** y la **búsqueda A*** para encontrar un camino desde el nodo **A** hasta el nodo **G**, y comparad las soluciones. *Es necesario detallar las búsquedas en vuestra solución (tenéis ejemplos en las soluciones de las PECs resueltas de otros semestres).*

¿Es admisible esta heurística? Aplicad el algoritmo de Dijkstra (¡que deberíais conocer!) para justificarlo. ¿Es consistente?

Solución:

El formato de las soluciones es un par de listas, línea en blanco, par de listas, línea en blanco, etc. Del par de listas, la primera es la lista de abiertos (o pendientes) y la segunda es la lista de cerrados (o explorados).

Búsqueda Ávida:

[['A', 12]]

[]

[['C', 8], ['B', 10]]

[['A', 12]]

[['F', 3], ['H', 4], ['D', 7], ['B', 10]]

[['C', 8], ['A', 12]]

['G', 0], ['H', 4], ['D', 7], ['B', 10]]

['F', 3], ['C', 8], ['A', 12]]

['fAtoC', 'fCtoF', 'fFtoG'] - La solución encontrada por la búsqueda àvida es incorrecta:

A → C → F → G de coste 13

Búsqueda A*:

['A', 12]]

[]

['C', 12.0], ['B', 13.0]]

['A', 12]]

['H', 11.0], ['F', 12.0], ['B', 13.0], ['D', 13.0]]

['C', 12.0], ['A', 12]]

['I', 11.0], ['F', 12.0], ['D', 13.0], ['B', 13.0]]

['H', 11.0], ['C', 12.0], ['A', 12]]

['F', 12.0], ['G', 12.0], ['B', 13.0], ['D', 13.0]]

['I', 11.0], ['H', 11.0], ['C', 12.0], ['A', 12]]

['G', 12.0], ['D', 13.0], ['B', 13.0], ['G', 13.0]]

['F', 12.0], ['I', 11.0], ['H', 11.0], ['C', 12.0], ['A', 12]]

['fAtoC', 'fCtoH', 'fHtoI', 'fIttoG'] - La solución encontrada por la búsqueda A* es óptima:

A → C → H → I → G de cost 12

¿Y cómo sabemos que es óptima? Comprobemos si h es admisible (deducimos h* usando el algoritmo de Dijkstra):

12 = h('A') ≤ h*('A') = 12 → **correcto**

10 = h('B') ≤ h*('B') = 11 → **correcto**

$$8 = h('C') \leq h^*('C') = 8 \rightarrow \text{correcto}$$

$$7 = h('D') \leq h^*('D') = 8 \rightarrow \text{correcto}$$

$$4 = h('E') \leq h^*('E') = 5 \rightarrow \text{correcto}$$

$$3 = h('F') \leq h^*('F') = 4 \rightarrow \text{correcto}$$

$$0 = h('G') \leq h^*('G') = 0 \rightarrow \text{correcto}$$

$$4 = h('H') \leq h^*('H') = 5 \rightarrow \text{correcto}$$

$$2 = h('I') \leq h^*('I') = 3 \rightarrow \text{correcto}$$

por tanto **h** sí es admisible y por sabemos que el **camino encontrado por A*** es el **óptimo**.

Ahora está por ver si es consistente, como pide el enunciado. Habrá que comprobar para cada arista que el heurístico no cambia tanto como el coste asociado:

$$2 = |h('A') - h('B')| \leq 3$$

$$3 = |h('D') - h('E')| \leq 3$$

$$4 = |h('A') - h('C')| \leq 4$$

$$4 = |h('E') - h('G')| \leq 6$$

$$3 = |h('B') - h('D')| \leq 4$$

$$2 = |h('E') - h('I')| \leq 2$$

$$6 = |h('B') - h('E')| \leq 6$$

$$3 = |h('F') - h('G')| \leq 4$$

$$1 = |h('C') - h('D')| \leq 2$$

$$2 = |h('H') - h('I')| \leq 2$$

$$5 = |h('C') - h('F')| \leq 5$$

$$2 = |h('I') - h('G')| \leq 3$$

$$4 = |h('C') - h('H')| \leq 3$$

Por tanto, **NO** es consistente.

Juegos:

6.- (2 puntos) Considérese un árbol de juego de profundidad 2 donde el nodo raíz es de tipo **MIN** (estamos en el turno del adversario que desea minimizar nuestro resultado).

La raíz tiene dos hijos: n1 y n2 (ambos de tipo **MAX**).

El nodo n1 tiene dos hojas con valores: 5 y 2 (de izquierda a derecha).

El nodo n2 tiene dos hojas con valores: 8 y 12 (de izquierda a derecha).

Se pide:

a) Aplicar el algoritmo α - β de izquierda a derecha. Indicar detalladamente qué hojas se visitan, cuáles se podan y cuál es el valor final de la raíz.

b) Justifica por qué se produce (o no) la poda en este caso, haciendo referencia a los valores α y β .

Solución:

a) Trazado del recorrido

Exploración de n1 (MAX):

- Visita la hoja 5.
- Visita la hoja 2.
- El valor de n1 se convierte en 5 (el máximo entre 5 y 2).

Actualización de la Raíz (MIN):

- La raíz recibe el valor 5 de n1.
- Dado que la raíz es MIN, ahora sabe que su valor final será como máximo 5. Establecemos el umbral $\beta = 5$

Exploración de n2 (MAX):

- Visita la primera hoja, que es el 8.
- Al ser n2 un nodo MAX, su valor será 8 o superior ($\alpha = 8$).

La poda

- El algoritmo compara el nuevo valor encontrado ($\alpha = 8$) con el umbral de la raíz ($\beta = 5$).
- Condición de poda: Dado que $\alpha(8) \geq \beta(5)$, se produce una poda inmediata.
- La raíz MIN sabe que elegirá el camino de n1 (que vale 5), porque el camino de n2 ya vale 8 y sólo puede subir más.

Valor final de la raíz: 5. Hojas visitadas: 5, 2, 8. Hojas podadas: 12.

b) Justificación Teórica

La poda se produce porque el nodo n2 (MAX) ya ha garantizado un valor (8) peor para la raíz (MIN) que la alternativa que ya tenía (5).

Matemáticamente: En un nodo MAX, la poda tiene lugar cuando su α es mayor o igual al β heredado de los antepasados MIN.

En este caso, $8 \geq 5$

Intuitivamente: "Si yo (MIN) ya tengo una opción que vale 5, no seguiré mirando una opción (MAX) que ya sé que vale 8 o más, porque nunca la elegiré."

Recursos

Para realizar esta PEC el material imprescindible son los temas 2, 3, 4 y 5 del Módulo 2.

Criterios de valoración

Indicado en los problemas

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC1 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **23 de Marzo** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.